

Parte 3 - Pattern

Pattern (Modelli) di Progettazione

Le architetture a microservizi utilizzano specifici *pattern* per affrontare le sfide della complessità, della comunicazione e della gestione delle transazioni distribuite. I principali pattern trattati sono: **Domain-Driven Design (DDD)**, **Circuit Breaker** e **Saga Pattern**.

Domain-Driven Design

È un approccio di progettazione software che allinea lo sviluppo alle problematiche aziendali, considerando i problemi come domini.

Il pattern DDD è considerato un'ottima scelta per la progettazione di architetture a microservizi, poiché aiuta a definire con precisione i confini dei servizi, la loro comunicazione e a gestire modelli complessi.

1. **Bounded Context (Contesto Delimitato)**: ogni microservizio dovrebbe avere il proprio contesto delimitato, che di fatto è il suo dominio specifico.
2. **Isolamento dei dati (Data Isolation)**: ogni microservizio deve gestire in modo indipendente il proprio accesso e la persistenza dei dati. Questo può comportare l'adozione di tecniche e motori di database diversi più adatti al contesto specifico.
3. **Decomposizione funzionale**: DDD agevola la suddivisione della problematica in microservizi basata sulle funzionalità di dominio, guidata dai requisiti di business.
4. **Linguaggio Ubiquo**: DDD favorisce un linguaggio condiviso tra team di sviluppo e analisti funzionali, migliorando la comunicazione e minimizzando i fraintendimenti sui requisiti specifici del dominio.

DDD è utile per comprendere la complessità del dominio e definire entità, oggetti valore e aggregazioni, ma si applica principalmente a microservizi complessi (non, ad esempio, a un microservizio CRUD).

Circuit Breaker

È un pattern di resilienza fondamentale nelle architetture distribuite.

In un ambiente distribuito, le chiamate a risorse remote (altri microservizi) possono fallire a causa di connessioni lente, timeout dovuti a sovraccarichi o server non disponibili.

Una strategia iniziale è il retry (riprovare la chiamata dopo un certo tempo). Tuttavia, se l'errore è dovuto a un guasto che impiega tempo per risolversi, il tentativo continuo di retry può creare un sovraccarico esponenziale del traffico verso il servizio non funzionante, agendo di fatto come un attacco DoS.

Il circuit breaker funge da barriera che impedisce temporaneamente l'esecuzione di operazioni con probabilità di riuscita molto bassa.

- **Vantaggio**: evita di bloccare le risorse del sistema chiamante (memoria, thread, connessioni al DB), aumentando la resilienza dell'applicazione.

Macchina a stati del circuit breaker

Il circuit breaker funge da proxy per le richieste, monitorando il numero degli errori recenti e decidendo se consentire il transito o bloccarlo. Questo proxy è implementato come una macchina a stati:

1. **Chiuso (Closed)**: lo stato normale. Il servizio è funzionante. Il proxy conta gli errori recenti.

- **Transizione:** se il numero di errori supera una certa soglia, il proxy passa allo stato **Aperto** e avvia un *timer*.
2. **Aperto (Open):** le richieste sono immediatamente bloccate e restituiscono un esito negativo al chiamante. **Non vengono inviate al servizio guasto.**
- **Transizione:** allo scadere del timer, il proxy passa allo stato Semiaperto.
3. **Semiaperto (Half-Open):** stato di test. Solo un numero limitato richieste può passare per verificare se il servizio si è ripreso.
- **Transizione 1 (Successo):** se questo poche richieste hanno tutte esito positivo, il proxy torna allo stato chiuso.
 - **Transizione 2 (Fallimento):** se almeno una richiesta va in errore, il proxy torna immediatamente allo stato aperto e riavvia il timer.

Saga Pattern

È un modello alternativo per la gestione delle transazioni distribuite e della consistenza dei dati in un'architettura a microservizi.

Nelle architetture a microservizi, l'atomicità tra operazioni che coinvolgono più microservizi non è garantita nativamente. Poiché ogni microservizio è proprietario dei suoi dati, non è possibile garantire le proprietà ACID se una transazione coinvolge più microservizi con i loro DB indipendenti.

Il protocollo del **2PC (Two-Phase Commit)** è un modello per transazioni distribuite, ma presenta diversi problemi che lo rendono inadatto alle architetture a microservizi:

- **Single Point of Failure (SPOF):** la presenza di un coordinatore centrale.
- **Lentezza:** il microservizio più lento determina le tempistiche di avanzamento globale.
- **Incompatibilità:** non è supportato nei database NoSQL, comuni nei microservizi.

Una **saga** è una **sequenza di transazioni locali** eseguito da ogni partecipante (microservizio).

- **Transazione Locale (Local Transaction):** l'operazione di avanzamento eseguita da ciascun microservizio, che aggiorna il proprio database locale.
- **Transazione di Compensazione (Compensating Transaction):** l'operazione necessaria a ripristinare lo stato precedente (annullare il lavoro svolto) in caso di fallimento di una transazione locale successiva. Le transazioni di compensazione sono idempotenti e riprovabili, garantendo una gestione automatica dell'annullamento.

La Saga garantisce che tutte le operazioni di avanzamento siano completate correttamente, oppure che vengano annullate, mantenendo lo stato globale del sistema coerente.

Tipi di transazioni nella Saga

All'interno di una Saga, le transazioni locali si dividono in tre tipi:

1. **Transazioni di avanzamento (Forward Transactions):** sono le normali transazioni locali che fanno progredire la saga e che prevedono sempre una transazione di compensazione.
2. **Transazione Pivot:** al massimo una. Se va a buon fine, la saga completa il suo avanzamento con successo. Può essere non compensabile/non ripetibile, l'ultima compensabile o la prima ripetibile.
3. **Transazioni riprovabili (Retriable Transactions):** seguono la transazione pivot e offrono garanzia di esito positivo (possono essere ripetute più volte).

Esempio di acquisto (flusso saga):

- **Avanzamento:** controllo stock → Aggiorna quantità → Inserisci ordine.
- **Pivot:** Conferma acquisto.
- **Riprovable:** invio e-mail di conferma.
- **Compensazione (in caso di fallimento prima del pivot):** Ripristinare stock ← Ripristinare quantità ← Annulla ordine.

Implementazione della Saga

Esistono due approcci principali per implementare il Saga Pattern:

a. Coreografia (Choreography)

- Nessun coordinatore centrale. I microservizi collaborano scambiandosi messaggi (o eventi), tipicamente tramite un **message broker**. Ogni microservizio sa esattamente quali operazioni eseguire in risposta a un dato evento.
- **Vantaggi:** non c'è un Single Point of Failure centralizzato, le responsabilità sono distribuite. Semplice e veloce da sviluppare in contesti con un numero ristretto di microservizi.
- **Svantaggi:** la complessità cresce velocemente all'aumentare dei microservizi. Il flusso di lavoro può diventare confuso e c'è il rischio di cicli tra le dipendenze.

b. Orchestrazione (Orchestration)

- Un servizio complementare, l'orchestratore, gestisce e controlla tutta la saga. L'orchestratore invia comandi ai microservizi e riceve messaggi/stati in risposta, tipicamente tramite un message broker.
- **Vantaggi:** visione centralizzata di tutta la saga. Adatto a contesti complessi con molti partecipanti. Aiuta a evitare cicli di dipendenza. La logica di business è centralizzata, rendendo la modifica e l'aggiunta di nuovi servizi più semplici (basta modificare l'orchestratore).
- **Svantaggi:** l'orchestratore è un SPOF (se va giù, tutte le saga si fermano). Il numero di interazioni è il doppio rispetto alla coreografia (messaggio al servizio per l'operazione e messaggio di stato dall'orchestratore).

Coreografia vs. Orchestrazione (Scelta)

La scelta dipende dal contesto applicativo. In generale:

- In un singolo contesto, con pochi microservizi e poche transazioni distribuite, si preferisce l'approccio a Coreografia.
- In tutti gli altri casi, si propende per l'approccio ad Orchestrazione.